```
(*
                            CS51 Lab 19
                    2-dimensional cellular automata
 *)
(*
                                SOLUTION
 *)

module G = Graphics ;;

    (************************************************************
    Do not change either of the two module type signatures in this
    file. Doing so will likely cause your code not to compile
    against our unit tests.
    ************************************************************)

(*.....................................................................
  Specifying automata

  A cellular automaton (CA) is a square grid of cells each in a
  specific state. The grid is destructively updated according to an
  update rule. The evolving state of the CA can be visualized by
  displaying the grid in a graphics window.

  In this implementation, a particular kind of automaton, with its
  state space and update function, is specified by a module satisfying
  the `AUT_SPEC` signature. *)

module type AUT_SPEC =
  sig
    (* Automaton parameters *)
    type state                (* the space of possible cell states *)
    val initial : state       (* the initial (default) cell state *)
    val grid_size : int       (* width and height of grid in cells *)
    val update : (state array array) -> int -> int -> state
                              (* update grid i j -- returns the new
                                 state for the cell at position `i, j`
                                 in `grid` *)
    val random_state : unit -> state
                              (* returns a random state *)
    (* Rendering parameters *)
    val name : string         (* a display name for the automaton *)
    val side_size : int       (* width and height of cells in pixels *)
    val cell_color : state -> G.color
```

```
                                (* color for each state *)
    val legend_color : G.color (* color to render legend *)
    val font : string option   (* optional font for legend as per
                                  Graphics module *)
    val render_frequency : int (* how frequently grid is rendered
                                  (in ticks) *)
  end ;;

(*.....................................................................
  Automata functionality

  Implementations of cellular automata provides for the following
  functionality:

  * A current (mutable) grid that can be modified (`replace_grid`).

  * Creating additional fresh grids (`fresh_grid`).

  * Initializing the graphics window in preparation for rendering
    (`graphics_init`).

  * Mapping the automaton's update function simultaneously over all
    cells of the current grid (`update_grid`).

  * Running the automaton using a particular update function, and
    rendering it as it evolves (`run_grid`).

  as codified in the `AUTOMATON` signature.

  Note the difference between the argument structure of functions in
  `life.ml`, where the grid is passed in as argument to several
  functions, as compared to the approach here, where there is a single
  mutable current grid that is updated by the functions, so that no
  grid argument is required.
 *)

module type AUTOMATON =
  sig
    (* state -- Possible states that a grid cell can be in *)
    type state

    (* grid -- 2D grid of cell states *)
    type grid = state array array

    (* current_grid -- The current grid of the appropriate size as per
       the spec, which is initialized with all cells being in the
```

```
91          initial state. It can be modified directly or by `replace_grid`
92          or updated with the automaton's update rule. *)
93      val current_grid : grid
94
95      (* fresh_grid () -- Returns a fresh grid of the appropriate size
96          as per the spec, initialized with all cells being in the
97          initial state. *)
98      val fresh_grid : unit -> grid
99
100     (* random_grid () -- Returns a fresh grid of the appropriate
101         size as per the spec, initialized with cells in random
102         states. *)
103     val random_grid : unit -> grid
104
105     (* replace_grid new_grid -- Destructively changes the current grid
106         (`current_grid`, above) to `new_grid`. *)
107     val replace_grid : grid -> unit
108
109     (* graphics_init () -- Initialize the graphics window to the
110         correct size and other parameters. Auto-synchronizing is off,
111         so our code is responsible for flushing to the screen upon
112         rendering. *)
113     val graphics_init : unit -> unit
114
115     (* update_grid () -- Updates the current grid one "tick" by
116         updating each cell simultaneously as per the CA's update
117         rule. *)
118     val update_grid : unit -> unit
119
120     (* run_grid grid -- Initializes the current grid with `grid` and
121         repeatedly updates it using the automaton's update rule rule,
122         rendering the grid state to the graphics window until a key is
123         pressed. Assumes graphics has been initialized with
124         `graphics_init`. *)
125     val run_grid : grid -> unit
126   end ;;
127
128 (*.....................................................................
129   Implementing automata based on a specification
130
131   Given an automaton specification (satisfying `AUT_SPEC`), the
132   `Automaton` functor delivers a module satisfying `AUTOMATON` that
133   implements the specified automaton. *)
134
135 module Automaton (Spec : AUT_SPEC)
136        : (AUTOMATON with type state = Spec.state) =
```

```
137   struct
138     type state = Spec.state
139     type grid = Spec.state array array
140
141     (* fresh_grid () -- See module type documentation *)
142     let fresh_grid (() : unit) : grid =
143       Array.make_matrix Spec.grid_size Spec.grid_size Spec.initial
144
145     (* current -- The current grid, which is destructively updated
146        after each tick; initialized with the initial state. *)
147     let current_grid =
148       fresh_grid ()
149
150     let random_grid () : grid =
151       let mat = fresh_grid () in
152       for i = 0 to Spec.grid_size - 1 do
153         for j = 0 to Spec.grid_size - 1 do
154           mat.(i).(j) <- Spec.random_state ()
155         done
156       done;
157       mat ;;
158
159     (* replace_grid -- See module type documentation *)
160     let replace_grid (new_grid : grid) : unit =
161       for i = 0 to Spec.grid_size - 1 do
162         for j = 0 to Spec.grid_size - 1 do
163           current_grid.(i).(j) <- new_grid.(i).(j)
164         done
165       done
166
167     (* graphics_init -- See module type documentation *)
168     let graphics_init () : unit =
169       G.open_graph "";
170       G.resize_window (Spec.grid_size * Spec.side_size)
171                       (Spec.grid_size * Spec.side_size);
172       (match Spec.font with
173        | None -> ()
174        | Some fontspec -> G.set_font fontspec);
175       G.auto_synchronize false
176
177     (* update_grid -- See module type definition. *)
178     let update_grid =
179       (* use a single static temp grid across invocations *)
180       let temp_grid = fresh_grid () in
181       fun () -> for i = 0 to Spec.grid_size - 1 do
182                   for j = 0 to Spec.grid_size - 1 do
```

```
183                     temp_grid.(i).(j) <- Spec.update current_grid i j
184                   done
185                 done;
186                 replace_grid temp_grid
187
188     (* render_grid grid legend -- Renders the `grid` to the already
189        initalized graphics window including the textual `legend` *)
190     let render_grid (grid : grid) (legend : string) : unit =
191       (* draw the grid of cells *)
192       for i = 0 to Spec.grid_size - 1 do
193         for j = 0 to Spec.grid_size - 1 do
194           G.set_color (Spec.cell_color grid.(i).(j));
195           G.fill_rect (i * Spec.side_size) (j * Spec.side_size)
196                       (Spec.side_size - 1) (Spec.side_size - 1)
197         done
198       done;
199       (* draw the legend *)
200       G.moveto (Spec.side_size * 2) (Spec.side_size * 2);
201       G.set_color Spec.legend_color;
202       G.draw_string legend;
203       (* flush to the screen *)
204       G.synchronize ()
205
206     (* run_grid -- See module type documentation *)
207     let run_grid (initial_grid : grid) : unit =
208       replace_grid initial_grid;
209       let tick = ref 0 in
210       render_grid current_grid (Printf.sprintf "%s: tick %d" Spec.name !tick);
211       ignore (G.read_key ());    (* pause at start until key is pressed *)
212       while not (G.key_pressed ()) do
213         update_grid ();
214         tick := succ !tick;
215         if !tick mod Spec.render_frequency = 0 then
216           render_grid current_grid (Printf.sprintf "%s: tick %d" Spec.name !tick);
217       done;
218       ignore (G.read_key ());    (* clear the keypress *)
219       ignore (G.read_key ());    (* await another keypress *)
220       G.close_graph () ;;
221   end ;;
```